Código calculadora com explicação

```
import tkinter as tk  # Importa o Tkinter com o apelido tk

# Esta função será executada quando o usuário clicar no botão Somar
def somar():
    # .get() lê o conteúdo das caixas como texto;
    # float() converte esse texto em número
    n1 = float(entrada1.get())
    n2 = float(entrada2.get())

    resultado = n1 + n2

    # Altera o texto do Label para mostrar a soma na janela
    lbl_resultado.config(text=f"Resultado: {resultado}")


# Cria a janela principal
janela = tk.Tk()
janela.title("Calculadora")
janela.geometry("350x250")  # Largura x altura, em pixels

# Cria o texto e a caixa de entrada do primeiro número
lbl_numero1 = tk.Label(janela, text="Primeiro número:")
lbl_numero1.pack()  # Posiciona o componente na janela

entrada1 = tk.Entry(janela)
entrada1.pack()

# Cria o texto e a caixa de entrada do segundo número
lbl_numero2 = tk.Label(janela, text="Segundo número:")
lbl_numero2.pack()

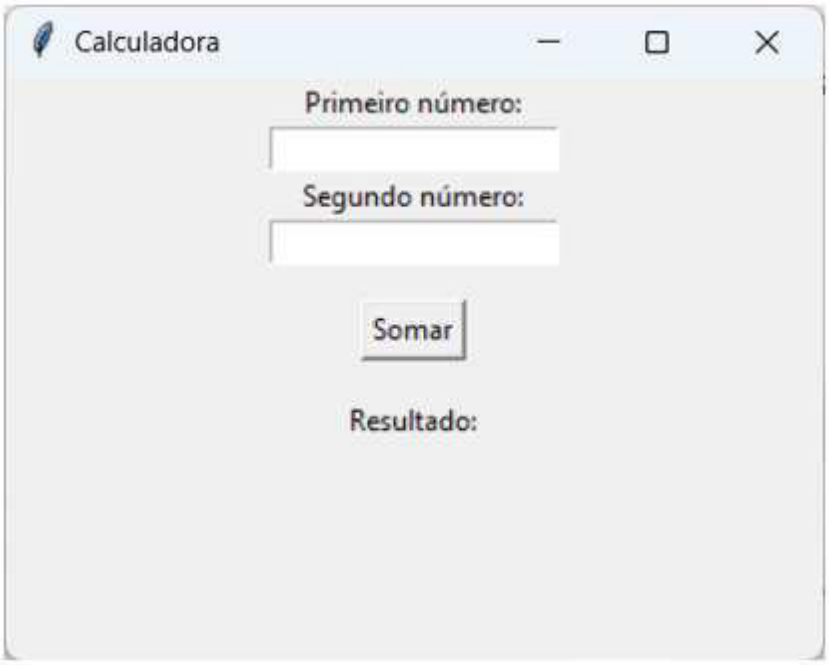
entrada2 = tk.Entry(janela)
entrada2.pack()

# Cria o botão e associa o clique à função somar
botao_somar = tk.Button(
    janela,
    text="Somar",
    command=somar  # Sem (): a função será chamada somente no clique
)
botao_somar.pack(pady=15)  # Adiciona espaço vertical ao redor do botão

# Cria o espaço onde o resultado será exibido
lbl_resultado = tk.Label(janela, text="Resultado:")
lbl_resultado.pack()

# Mantém a janela aberta e aguardando as ações do usuário
janela.mainloop()
```

O resultado visual será aproximadamente:



Construindo uma calculadora com Tkinter

Anteriormente, criamos uma interface com duas caixas de entrada e um botão **Somar**. Agora vamos aproximar a interface de uma calculadora: os números serão digitados pelos **botões da própria janela**.

O objetivo desta etapa é aprender a organizar botões em linhas e colunas, controlar o visor e associar ações diferentes aos cliques. Ao final, você desenvolverá a lógica das operações.

Organizando componentes com grid()

No primeiro programa, usamos pack() para colocar um componente abaixo do outro. Para montar o teclado de uma calculadora, precisamos controlar **a linha e a coluna** de cada botão. Para isso, usamos grid().

```
import tkinter as tk

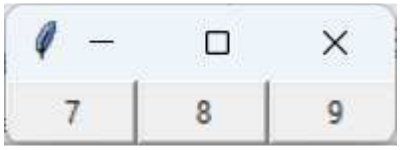
janela = tk.Tk()
janela.title("Teclado da calculadora")

botao_7 = tk.Button(janela, text="7", width=6)
botao_7.grid(row=0, column=0)

botao_8 = tk.Button(janela, text="8", width=6)
botao_8.grid(row=0, column=1)

botao_9 = tk.Button(janela, text="9", width=6)
botao_9.grid(row=0, column=2)

janela.mainloop()
```



➢ row indica a **linha**.

➢ column indica a **coluna**.

A contagem começa em **zero**.

Atenção: use grid() para todos os componentes colocados diretamente em janela neste exemplo. Não misture pack() e grid() no mesmo espaço que contém os componentes.

Criando o visor

Um `Entry` pode funcionar como o visor da calculadora. Podemos ler, inserir e apagar seu conteúdo:

Comando	Ação
`visor.get()`	Lê o conteúdo do visor.
`visor.insert(tk.END, "7")`	Insere `7` no fim do conteúdo.
`visor.delete(0, tk.END)`	Apaga todo o conteúdo.
`visor.delete(len(visor.get()) - 1, tk.END)`	Apaga o último caractere, se houver conteúdo.

Observe uma janela que já possui visor e os botões numéricos:

```
import tkinter as tk

def digitar_7():
    visor.insert(tk.END, "7")

def digitar_8():
    visor.insert(tk.END, "8")

def limpar():
    visor.delete(0, tk.END)

janela = tk.Tk()
janela.title("Primeiros botões")
```




```
# O visor ocupa três colunas da primeira linha
visor = tk.Entry(janela, width=24, justify="right")
visor.grid(row=0, column=0, columnspan=3, padx=5, pady=5)

botao_7 = tk.Button(janela, text="7", width=6, command=digitar_7)
botao_7.grid(row=1, column=0, padx=2, pady=2)

botao_8 = tk.Button(janela, text="8", width=6, command=digitar_8)
botao_8.grid(row=1, column=1, padx=2, pady=2)

botao_c = tk.Button(janela, text="C", width=6, command=limpar)
botao_c.grid(row=1, column=2, padx=2, pady=2)

janela.mainloop()
```

Aqui aparecem dois detalhes novos:

- `columnspan=3` faz o visor ocupar o espaço de **três colunas**.
- `justify="right"` alinha o conteúdo do visor à direita.

Teste: clique em `7`, depois em `8` e, por fim, em `C`. O visor deve mostrar `78` antes de ser limpo.

Uma função pode atender a vários botões

Criar uma função quase igual para cada dígito gera repetição. Podemos receber o dígito como argumento:

```
def digitar(numero):
    visor.insert(tk.END, numero)
```

Para associar o botão a essa função, usamos uma pequena função anônima, chamada `lambda`:

```
botao_7 = tk.Button(
    janela,
    text="7",
    command=lambda: digitar("7")
)
```

Nesse caso, `lambda` significa: **quando o botão for clicado, chame** `digitar("7")`.
Compare:

```
command=limpar                  # Função sem argumento
command=lambda: digitar("7")    # Função que receberá um argumento
```

Teste de compreensão
Se criarmos o botão abaixo, o que deverá aparecer no visor após o clique?

```
tk.Button(janela, text="4", command=lambda: digitar("4"))
```

Que informações a calculadora precisa guardar?

Os botões numéricos resolvem apenas parte do problema. Para calcular `8 + 3`, o programa precisa acompanhar:

1. O **primeiro número**: `8`.
2. A **operação escolhida**: `+`.
3. O **segundo número**: `3`.

Quando o usuário clicar em =, uma função deverá verificar qual operação foi escolhida, calcular o resultado e mostrá-lo no visor.

Pense na sequência de eventos:

Clique	O que o programa precisa fazer
8	Mostrar `8` no visor.
+	Guardar `8` e a operação; preparar o visor para o próximo número.
3	Mostrar `3` no visor.
=	Calcular `8 + 3` e mostrar `11`.

Essa lógica será desenvolvida por você no exercício. Use funções, variáveis e estruturas de decisão já estudadas. **Não utilize** `eval()`: implemente cada operação no seu próprio código.

Exercício — Calculadora gráfica

Uma empresa deseja substituir uma pequena calculadora feita para o console por uma versão com interface gráfica. Você deverá criar, em Python e Tkinter, uma calculadora com visor e teclado visual, inspirada na disposição dos botões da calculadora do Windows.

Requisitos da interface
A janela deverá conter:

- um visor criado com `Entry`;
- botões para os dígitos de `0` a `9`;
- botão para o separador decimal `.`;
- botões para adição `+`, subtração `−`, multiplicação `×` e divisão `÷`;
- botão `=` para mostrar o resultado;
- botão `C` para limpar a operação;
- botão `⌫` para apagar o último caractere digitado.



Organize os botões em linhas e colunas com `grid()`. A aparência não deve ser idêntica à do Windows, mas o teclado deve ser fácil de entender e utilizar.

Regras de funcionamento

1. Os números devem ser inseridos no visor ao clicar nos botões.
2. O botão decimal não deve permitir dois pontos no **mesmo número**.
3. Ao escolher uma operação, o programa deve guardar o primeiro número e a operação selecionada.
4. O botão = deve calcular usando o segundo número e mostrar o resultado no visor.
5. O botão `C` deve limpar o visor **e** os dados guardados da operação.
6. O botão `⌫` deve apagar apenas o último caractere do número que está sendo digitado.
7. Uma tentativa de divisão por zero deve mostrar uma mensagem compreensível na interface, sem encerrar o programa.
8. O programa não deve encerrar se o usuário pressionar = sem ter informado os dois números.

Escopo desta versão: cada cálculo terá **dois números e uma operação**. Após obter o resultado, o usuário poderá pressionar `C` para iniciar outro cálculo. Sequências como `2 + 3 × 4`, histórico e memória ficam fora deste exercício.

Testes que deverá realizar

Botões pressionados	Resultado esperado
`8 + 3 =`	`11`
`9 − 4 =`	`5`
`2.5 × 4 =`	`10`
`8 ÷ 2 =`	`4`
`8 ÷ 0 =`	Mensagem de divisão por zero; programa continua aberto.
`1, 2 ⌫`	Visor mostra `1`.
`7, C`	Visor fica limpo e a operação anterior é descartada.

Correção

1. Tudo funcionando e respondendo corretamente a uma pergunta
2. Tudo funcionando e respondendo corretamente a duas pergunta, e sendo criativo na questão de visual da interface gráfica
3. Tudo funcionando e respondendo corretamente a três pergunta, e sendo criativo na questão de visual da interface gráfica, apresenta no mínimo uma propriedade e uma função nova, que não foi apresentada em aula.